Київський національний університет імені Тараса Шевченка
Кафедра мережевих і Інтернет технологій

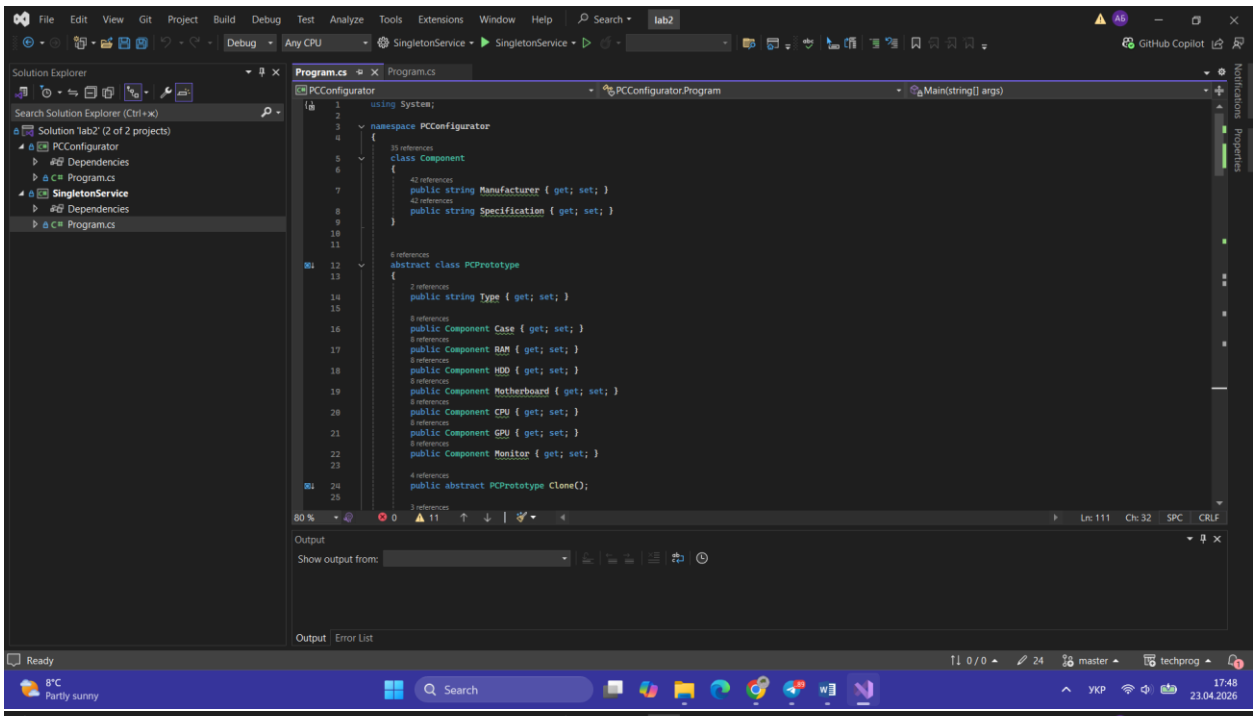
Лабораторна робота №2
з дисципліни
«Технології створення програмних продуктів»

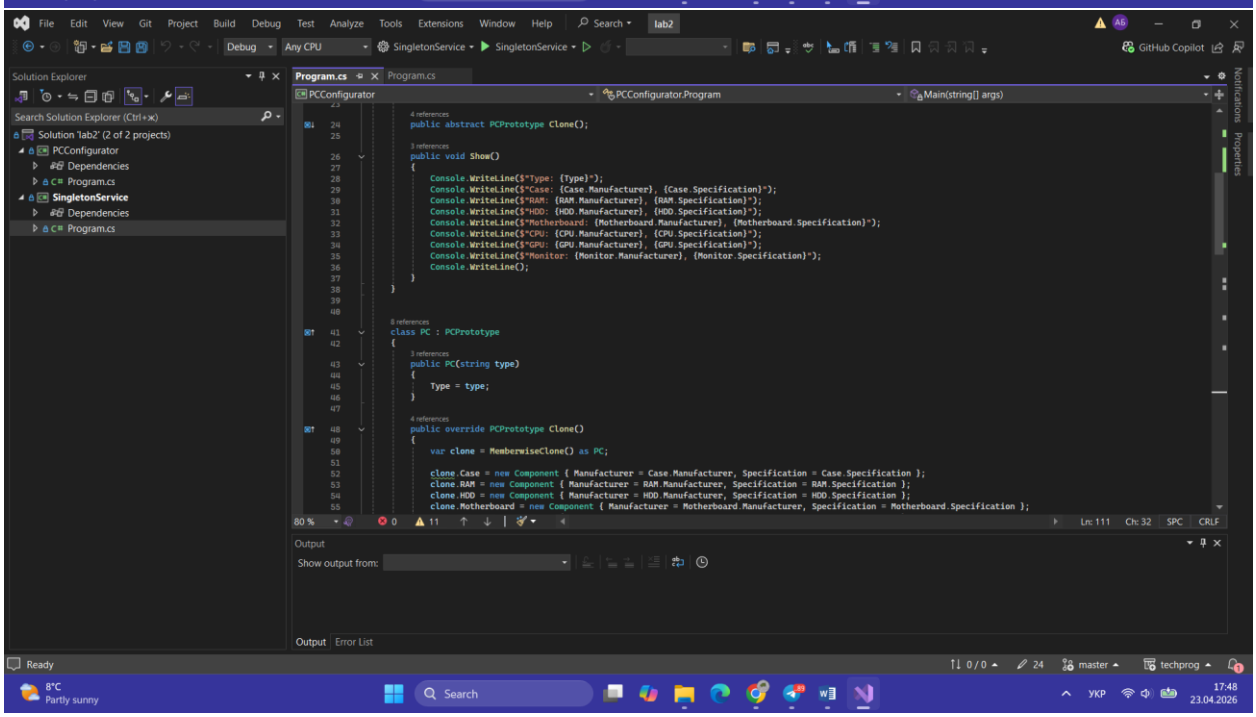
ШАБЛОН PROTOTYPE ТА ШАБЛОН SINGLETON

Група: МІТ-м11
Студентка: Булгакова Анна

Київ 2026

Завдання 1. Використовуючи шаблон Prototype, реалізувати програму PCConfigurator, яка б дозволяла створювати екземпляри класів – персональних комп’ютерів для домашнього, ігрового та офісного використання, із наперед заданими значеннями властивостей для характеристик основних компонентів: корпус, оперативна пам’ять, HDD, системна плата, процесор, відеоадаптер, монітор. Кожен компонент повинен мати обов’язкову характеристику Виробник та одну або більше специфічних саме для свого типу компонентів додаткові характеристики.



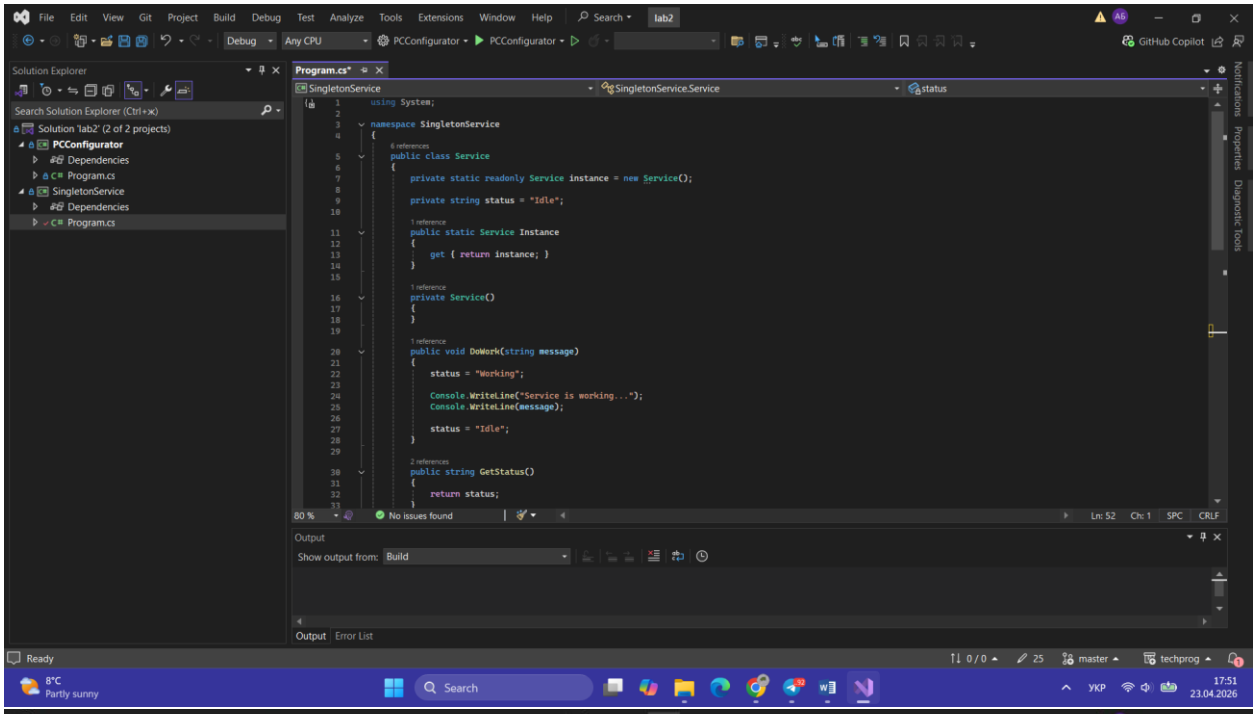


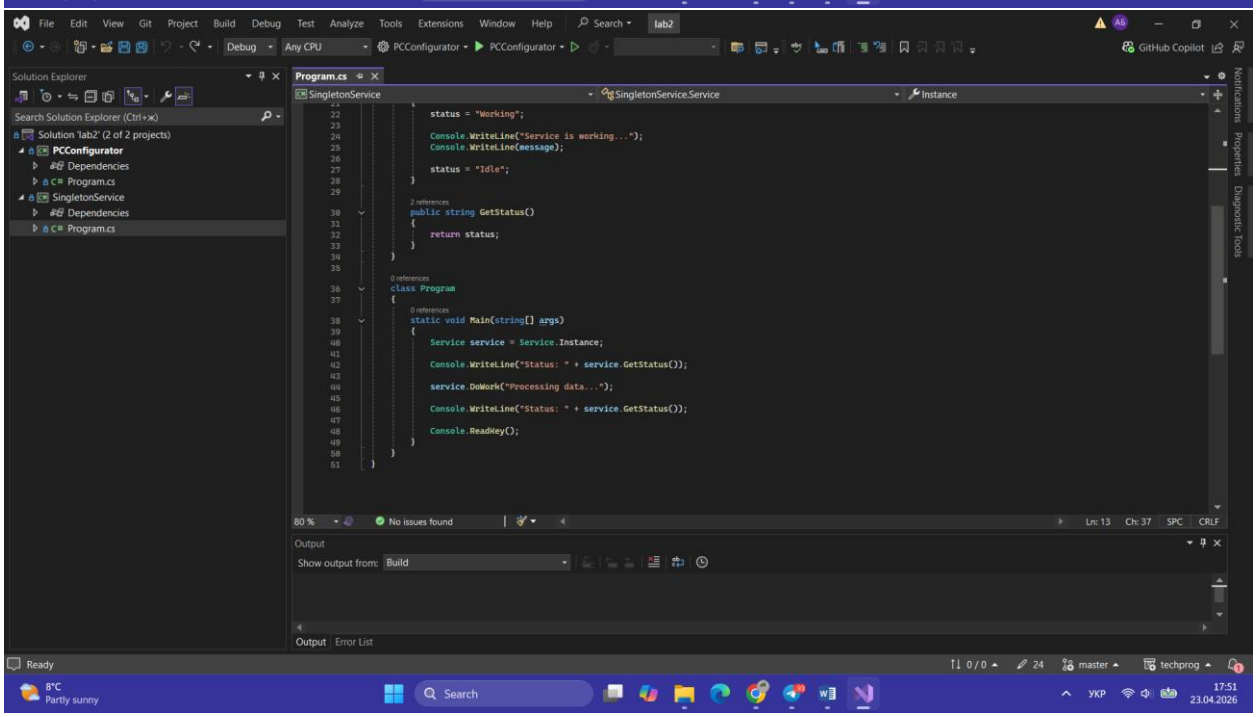
```
            clone.Motherboard = new Component { Manufacturer = Motherboard.Manufacturer, Specification = Motherboard.Specification };
            clone.CPU = new Component { Manufacturer = CPU.Manufacturer, Specification = CPU.Specification };
            clone.GPU = new Component { Manufacturer = GPU.Manufacturer, Specification = GPU.Specification };
            clone.Monitor = new Component { Manufacturer = Monitor.Manufacturer, Specification = Monitor.Specification };

            return clone;
        }
    }

    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {

            PC homePC = new PC("Home PC")
            {
                Case = new Component { Manufacturer = "CoolerMaster", Specification = "Mini Tower" },
                RAM = new Component { Manufacturer = "Kingston", Specification = "8GB" },
                HDD = new Component { Manufacturer = "Seagate", Specification = "1TB HDD" },
                Motherboard = new Component { Manufacturer = "ASUS", Specification = "B450" },
                CPU = new Component { Manufacturer = "AMD", Specification = "Ryzen 3" },
                GPU = new Component { Manufacturer = "Integrated", Specification = "Vega Graphics" },
                Monitor = new Component { Manufacturer = "LG", Specification = "22 inch" }
            };


            PC gamingPC = new PC("Gaming PC")
            {
                Case = new Component { Manufacturer = "NZXT", Specification = "Gaming Tower" },
                RAM = new Component { Manufacturer = "Corsair", Specification = "32GB" },
                HDD = new Component { Manufacturer = "Samsung", Specification = "1TB SSD" },
                Motherboard = new Component { Manufacturer = "MSI", Specification = "Z790" },
                CPU = new Component { Manufacturer = "Intel", Specification = "Core i9" },
                GPU = new Component { Manufacturer = "NVIDIA", Specification = "RTX 4080" },
                Monitor = new Component { Manufacturer = "ASUS", Specification = "27 inch 144Hz" }
```

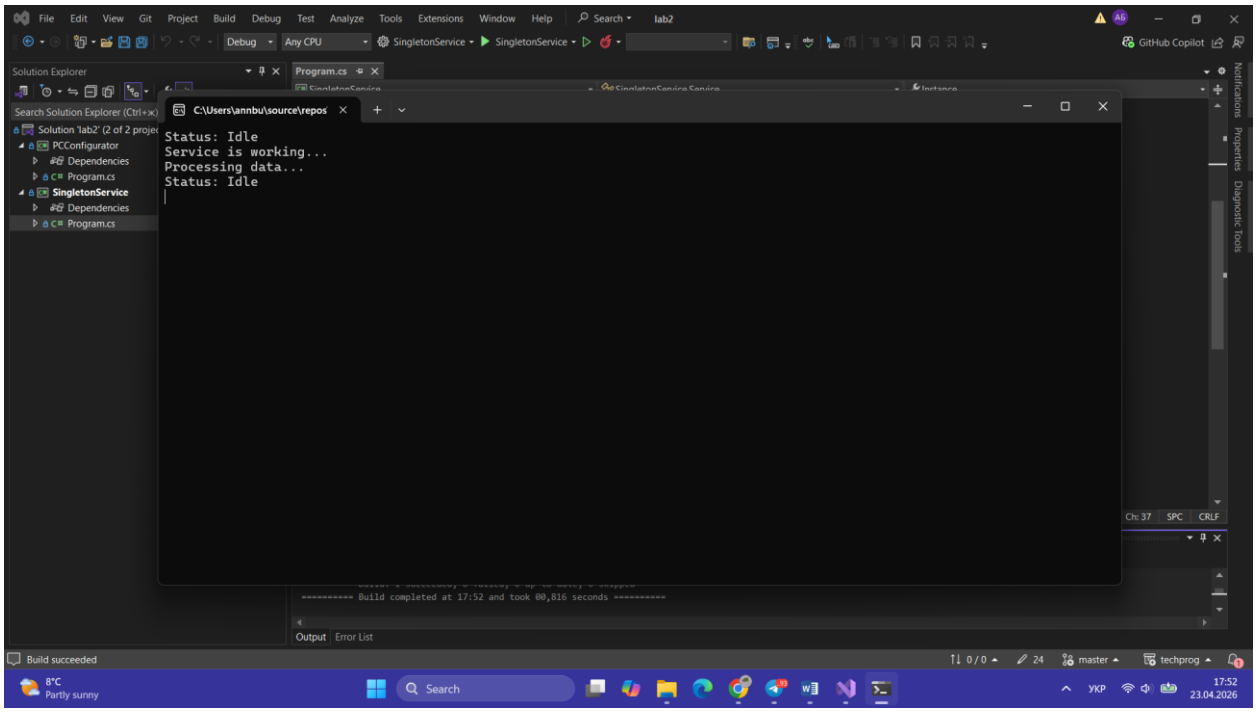
```
                Case = new Component { Manufacturer = "NZXT", Specification = "Gaming Tower" },
                RAM = new Component { Manufacturer = "Corsair", Specification = "32GB" },
                HDD = new Component { Manufacturer = "Samsung", Specification = "1TB SSD" },
                Motherboard = new Component { Manufacturer = "MSI", Specification = "Z790" },
                CPU = new Component { Manufacturer = "Intel", Specification = "Core i9" },
                GPU = new Component { Manufacturer = "NVIDIA", Specification = "RTX 4080" },
                Monitor = new Component { Manufacturer = "ASUS", Specification = "27 inch 144Hz" }
            };


            PC officePC = new PC("Office PC")
            {
                Case = new Component { Manufacturer = "Dell", Specification = "Compact Case" },
                RAM = new Component { Manufacturer = "Kingston", Specification = "16GB" },
                HDD = new Component { Manufacturer = "WD", Specification = "512GB SSD" },
                Motherboard = new Component { Manufacturer = "Gigabyte", Specification = "B460" },
                CPU = new Component { Manufacturer = "Intel", Specification = "Core i5" },
                GPU = new Component { Manufacturer = "Integrated", Specification = "Intel UHD" },
                Monitor = new Component { Manufacturer = "Samsung", Specification = "24 inch" }
            };

            Console.WriteLine("Prototype PCs created.\n");

            PCPrototype newHomePC = homePC.Clone();
            PCPrototype newGamingPC = gamingPC.Clone();
            PCPrototype newOfficePC = officePC.Clone();

            newHomePC.Show();
            newGamingPC.Show();
            newOfficePC.Show();

            Console.ReadKey();
        }
    }
}
```

```
Prototype PCs created.

Type: Home PC
Case: CoolerMaster, Mini Tower
RAM: Kingston, 8GB
HDD: Seagate, 1TB HDD
Motherboard: ASUS, B450
CPU: AMD, Ryzen 3
GPU: Integrated, Vega Graphics
Monitor: LG, 22 inch

Type: Gaming PC
Case: NZXT, Gaming Tower
RAM: Corsair, 32GB
HDD: Samsung, 1TB SSD
Motherboard: MSI, Z790
CPU: Intel, Core i9
GPU: NVIDIA, RTX 4080
Monitor: ASUS, 27 inch 144Hz

Type: Office PC
Case: Dell, Compact Case
RAM: Kingston, 16GB
HDD: WD, 512GB SSD
Motherboard: Gigabyte, B460
CPU: Intel, Core i5
GPU: Integrated, Intel UHD
Monitor: Samsung, 24 inch
```

Завдання 2. Реалізуйте клас Service, який буде використовувати шаблон Singleton для створення єдиного об’єкта сервісу. Клас Service повинен мати метод DoWork(), який приймає рядок в якості параметра і виводить його на консоль. Клас Service також повинен мати метод GetStatus(), який повертає рядок «Working» або «Idle» в залежності від того, чи виконується в даний момент метод DoWork() чи ні.







Висновки: На даній лабораторній роботі було ознайомлено із шаблонами проєктування **Prototype** (шаблон, що породжує класи) та **Singleton** (шаблон, що породжує об’єкти). Було досліджено принцип створення об’єктів шляхом клонування на основі шаблону Prototype, що дозволяє уникнути складного процесу ініціалізації та забезпечує ефективне створення об’єктів із заданими параметрами. У практичній частині реалізовано систему конфігурації персональних комп’ютерів із використанням прототипів. Також було розглянуто шаблон Singleton, який гарантує існування лише одного екземпляра класу та надає до нього глобальну точку доступу. У процесі виконання завдання було створено сервіс, що використовує цей підхід для контролю свого стану. Отже, застосування шаблонів Prototype та Singleton дозволяє оптимізувати процес створення об’єктів і забезпечити контроль над кількістю екземплярів класів, що підвищує ефективність та надійність програмного забезпечення.